Lab13(TCL)

Task-1

At the beginning of each semester, students register for courses. Sometimes they select the wrong course, sometimes seats fill up, and sometimes operators make typing mistakes. To avoid corrupt or incorrect records, the Course Registration System uses transactions to ensure only correct registrations are saved.

Create table:

```
CREATE TABLE Course(
 CourseID INT PRIMARY KEY,
 CourseName VARCHAR(50),
 SeatsAvailable INT
);
CREATE TABLE Registration(
 RegID INT PRIMARY KEY,
 StudentName VARCHAR(50),
 CourseID INT,
 FOREIGN KEY (CourseID) REFERENCES Course(CourseID)
);
#Insert Initial Data
```

Insert Initial Data:

```
INSERT INTO Course VALUES
(101, 'Database Systems', 3),
(102, 'Operating Systems', 2);
```

Correct Registration → COMMIT:

```
BEGIN TRAN;
INSERT INTO Registration VALUES (1, 'Hassan Ali', 101);
UPDATE Course SET SeatsAvailable = SeatsAvailable - 1 WHERE CourseID = 101;
COMMIT;
```

Wrong Registration → ROLLBACK

```
BEGIN TRAN;
INSERT INTO Registration VALUES (2, 'Mehwish Khan', 102);
UPDATE Course SET SeatsAvailable = SeatsAvailable - 1 WHERE CourseID = 102;
-- Mistake found: wrong course selected
ROLLBACK;
```

SAVEPOINT:

```
BEGIN TRAN;
INSERT INTO Registration VALUES (3, 'Ali Raza', 101);
UPDATE Course SET SeatsAvailable = SeatsAvailable - 1 WHERE CourseID = 101;
SAVE TRAN HalfWay;
INSERT INTO Registration VALUES (4, 'Fatima Noor', 102);
UPDATE Course SET SeatsAvailable = SeatsAvailable - 1 WHERE CourseID = 102;
-- Student 4 wants to cancel
ROLLBACK TRAN HalfWay;
COMMIT;
```

Task-2:

You are working as a database engineer for ShopMax.pk, an online shopping platform. When a customer places an order, many things happen:

1. Customer record is validated
2. Items are checked for stock availability
3. Stock quantity is reduced
4. Payment is deducted
5. Order is created
6. Invoice is generated

Create tables:

```
CREATE TABLE Customers(
 CustomerID INT PRIMARY KEY,
 CustomerName VARCHAR(50),
 WalletBalance DECIMAL(10,2)
);
CREATE TABLE Products(
 ProductID INT PRIMARY KEY,
 ProductName VARCHAR(50),
 UnitPrice DECIMAL(10,2),
 StockQty INT
);
CREATE TABLE Orders(
 OrderID INT PRIMARY KEY,
 CustomerID INT,
 OrderDate DATETIME,
 TotalAmount DECIMAL(10,2),
 FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
);
```

Insert Initial Data

```
INSERT INTO Customers VALUES
(1, 'Zara Ahmed', 30000),
(2, 'Hassan Raza', 15000);
INSERT INTO Products VALUES
(11, 'Laptop', 85000, 5),
(12, 'Headphones', 3000, 25),
(13, 'Keyboard', 2500, 10);
```

Start Transaction & Validate Customer

```
BEGIN TRAN;
-- Step 1: Create New Order
INSERT INTO Orders VALUES (1001, 1, GETDATE(), 0);
-- Savepoint after order creation
SAVE TRAN AfterOrderCreated
```

Add Order Items (Multiple Products)

```
-- Add first product (Laptop)
INSERT INTO OrderItems VALUES (201, 1001, 11, 1, 85000);
UPDATE Products SET StockQty = StockQty - 1 WHERE ProductID = 11;
SAVE TRAN AfterLaptop;
-- Add second product (Headphones)
INSERT INTO OrderItems VALUES (202, 1001, 12, 2, 3000);
UPDATE Products SET StockQty = StockQty - 2 WHERE ProductID = 12;
SAVE TRAN AfterHeadphones;
```

Calculate Total Amount

```
DECLARE @Total DECIMAL(10,2);
SELECT @Total = SUM(Quantity * ItemPrice)
FROM OrderItems
WHERE OrderID = 1001;
UPDATE Orders
SET TotalAmount = @Total
WHERE OrderID = 1001;
```

Deduct Payment From Customer Wallet

```
⇓ If wallet balance < total → FULL ROLLBACK
DECLARE @CustomerBalance DECIMAL(10,2);
SELECT @CustomerBalance = WalletBalance FROM Customers WHERE
CustomerID = 1;
IF (@CustomerBalance < @Total)
BEGIN
 PRINT 'Payment Failed! Insufficient Balance.';
 ROLLBACK; -- FULL ROLLBACK
 RETURN;
END
-- Deduct payment
UPDATE Customers
SET WalletBalance = WalletBalance - @Total
WHERE CustomerID = 1;
#FINAL COMMIT
COMMIT;
PRINT 'Order Processed Successfully!';
```